

Addressing and static routing

Follow-along: build the network until every host reaches every host

Overview

Definition: Private address space

The three blocks RFC 1918 reserves for networks that are not part of the public internet: 10.0.0.0/8, 172.16.0.0/12 and 192.168.0.0/16. Anyone may number a network out of them without asking a registry, so the same address is in use at thousands of sites at once. The price is that the addresses have no global meaning: routing information about them is not passed between organisations, and packets carrying them are not forwarded across those links. This lab numbers its three subnets out of the third block, as 192.168.1.0/24, 192.168.2.0/24 and 192.168.3.0/24.

Nothing in this lab attacks anything, and nothing in it defends against an attack. What it hands you is a network with no configuration on it. Eleven containers are running and wired into three subnets out of **private address space**, and that is the whole of what you are given: not one interface carries an address, not one interface is switched on, and not one device holds a route. You give each of the ten interfaces its address, give each host a first hop off its own subnet, and give each of the two routers the one route it is missing, until all six hosts reach each other. The oracle is the last of those: thirty ordered pairs of hosts, all thirty reachable.

Every route in this lab is one you type. Nothing here runs a routing protocol, so no router learns anything from any other router, and no route changes unless you change it. What this lab builds by hand is the thing a routing protocol exists to fill in: the routing table itself, and what a device does when it looks a destination up in that table and finds nothing.

Running this lab in the TUI

You drive the lab from the MiniLabs launcher. Clone it if the machine does not have it, and pull if it does, because handouts and lab scripts change together:

```
git clone https://github.com/nzRichie/MiniLab.git ~/MiniLab    # first time
cd ~/MiniLab && git pull                                       # already cloned
```

Start the launcher from that directory:

```
cd ~/MiniLab && ./minilabs
```

If it exits with a Docker error, follow the setup in the repository's `README.md` and run it again.

The root menu lists one category per row. Select **Network configuration**, then **Addressing and static routing**, to reach this lab's actions:

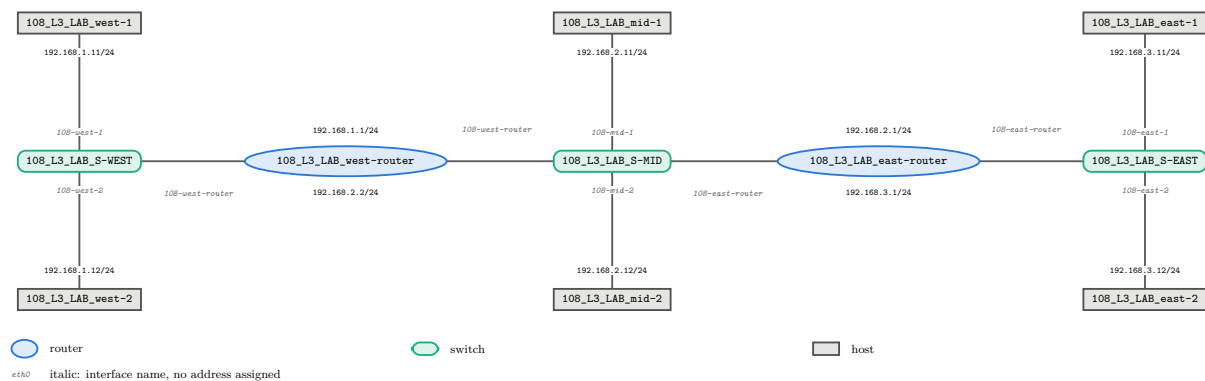
- **Spawn** creates this lab's containers and wiring. Wait for the panel to turn green before you go on. The first spawn of a lab also builds its images, which takes a few minutes; later spawns take seconds.
- **Status** prints the lab's state and its success oracle: which interfaces carry which address, which hosts reach which other hosts, and how much of the network is still to build. Status

takes over the whole screen: the arrow keys scroll its output, and Enter on **Done** closes it.

- **Shell command for a node** lists every node (**west-1**, **west-2**, **mid-1**, **mid-2**, **east-1**, **east-2**, **west-router**, **east-router**) and the `docker exec -it ...` line that opens a shell on it. Copy the line for the node a step below names into your own terminal. It holds the screen until Enter on **Done**, as Status does.
- **Reset to baseline** deletes every address and every route you configured and puts the network back in the unconfigured state it spawned in, without tearing it down.
- **Teardown** removes the lab.
- **Open handout** opens this document.

Spawn the lab now, then work through Part 1 in your own terminal. You will want several terminals open at once, because some steps run a capture on one machine while you send traffic from another. **Back** (or Esc) returns to the lab list.

The setup



Three subnets, six hosts, two Linux routers and three switches. West Net and East Net each hold two hosts and one router. Middle Net holds two hosts and *both* routers, which is what joins the two ends of the network together. Each node is labelled with the container name you pass to `docker exec`, and each end of a link with the address that end is meant to end up with. None of those addresses is configured yet.

Every container in the table below is named `108_L3_LAB_` followed by the device name in its first column, so `west-1` is `108_L3_LAB_west-1`.

Device	Interface	Address	Subnet
west-1	108-WEST	192.168.1.11	West Net 192.168.1.0/24
west-2	108-WEST	192.168.1.12	West Net
west-router	108-WEST	192.168.1.1	West Net
	108-MID	192.168.2.2	Middle Net 192.168.2.0/24
mid-1	108-MID	192.168.2.11	Middle Net
mid-2	108-MID	192.168.2.12	Middle Net
east-router	108-MID	192.168.2.1	Middle Net
	108-EAST	192.168.3.1	East Net 192.168.3.0/24
east-1	108-EAST	192.168.3.11	East Net
east-2	108-EAST	192.168.3.12	East Net

An interface is named after the subnet it sits on rather than after the machine it is in, so every

device on West Net calls its lab interface 108-WEST. A router holds two of these, one per subnet, and that is the only way to tell its two interfaces apart from inside the container. The 108 in every name is this lab's AS number; other labs in the catalogue use their own.

Definition: Broadcast domain

The set of interfaces a broadcast frame reaches. A Layer 2 switch floods a broadcast frame out of every port except the one it arrived on, so everything plugged into one of this lab's switches is in one broadcast domain and can exchange frames without a router. That is what makes a broadcast domain the natural boundary of an IP subnet, and it is why the three switches in the figure carry no addresses of their own: a switch forwards on MAC addresses and never looks at the IP header.

Each subnet is one Open vSwitch switch and therefore one **broadcast domain**. You never configure the switches; they are already forwarding, and the ports facing your devices are already up. Everything you configure is inside the eight host and router containers.

Two settings are already in place on the routers and cannot be changed from inside the container, because Docker mounts `/proc/sys` read-only there. Both routers are started with `net.ipv4.ip_forward=1`, which is what makes them forward packets addressed to somebody else rather than discard them, and with `net.ipv4.conf.all.send_redirects=1`, which Part 5 is about. A `sysctl -w` of either one inside a container is refused; that is the container, not a mistake you made.

The tooling is the standard Linux networking set, and you will use four commands for almost everything:

- `ip addr`: read and set the addresses on an interface.
- `ip link`: read and set whether an interface is switched on.
- `ip route`: read and set the routing table, and with `ip route get`, ask what the kernel would do with one specific destination right now.
- `ping`: send an ICMP echo request and report what came back, or what error came back instead.

`tcpdump` is there for the two steps that read packets off the wire, and `route -n` and `ifconfig` are installed so you can see the older commands print the same information in different columns. Open a shell wherever a step says so with:

```
docker exec -it <container> bash
```

Assessment. This lab is graded on eighteen questions, marked inline below as **Question *N***, each placed at the step whose output it asks about. Answer all eighteen in a separate document as you work through the steps. Every question is answered from what you see on screen at that step, or from a prediction you make before running the next command.

Part 1: two hosts on one subnet

Nothing in a network works until two machines on the same wire can exchange a packet, and that takes two things on each of them: an address that says which subnet the machine is on, and an interface that is switched on. Part 1 does that for west-1 and west-2 and nothing else. The other nine containers stay unconfigured until Part 2.

1. **Look at what you have been given.** Open a shell on west-1 and read its three tables before you change anything:

```
# 108_L3_LAB_west-1: the first host you configure
docker exec -it 108_L3_LAB_west-1 bash
ip link show           # 108-WEST exists, and says DOWN
ip addr show           # 108-WEST has no inet line
ip route show          # prints nothing at all
```

An empty `ip route show` is the normal state of a machine with no configured interface. The routing table is not a file that ships with entries in it; nothing in this lab runs a routing daemon, so every entry in it was either put there by a command or added by the kernel as a side effect of one.

2. **Give west-1 its address and switch the interface on.** The address goes on with its prefix length attached, in the `a.b.c.d/length` form:

```
# 108_L3_LAB_west-1
ip addr add 192.168.1.11/24 dev 108-WEST
ip link set dev 108-WEST up
ip addr show dev 108-WEST    # check both the address and the /24
```

Definition: Connected route

The routing table entry the kernel creates by itself when you give an interface an address, one per address. It covers the whole subnet the prefix length describes and names no gateway, because every address in that subnet is reachable by putting a frame on the wire. `ip route show` prints it with `proto kernel scope link`, which is how you tell an entry you did not type from one you did.

3. **Read the routing table again.** It is no longer empty. The **connected route** that appeared is the reason the prefix length is part of the address and not an optional extra:

```
# 108_L3_LAB_west-1
ip route show
route -n           # the same table, older command, different
                   columns
```

Question 1. Give the one route `ip route show` now prints on west-1. You typed no `ip route` command, so state which part of your `ip addr add` argument decided the route's destination, and which part decided how many addresses it covers.

4. **Configure west-2 the same way**, in a second terminal:

```
# 108_L3_LAB_west-2: the second host on West Net
docker exec -it 108_L3_LAB_west-2 bash
ip addr add 192.168.1.12/24 dev 108-WEST
ip link set dev 108-WEST up
```

Check that the two machines hold different addresses and the same prefix length. Two hosts

with the same address, or with prefix lengths that disagree about where the subnet ends, is the most common way this step goes wrong.

5. **Watch the first packet.** Before west-1 can put a frame on the wire for 192.168.1.12, it has to learn which hardware address that is. Empty both neighbour tables, start a capture on west-2, and then ping from west-1:

```
# 108_L3_LAB_west-2: leave this running
ip neigh flush all
tcpdump -nevi 108-WEST arp
```

```
# 108_L3_LAB_west-1: in your other terminal
ip neigh flush all
ping -n -c 1 192.168.1.12
```

Question 2. Give the number of ARP packets the capture on west-2 shows for that single ping. For each one, say which host sent it, whether its Ethernet destination address was the broadcast address or one specific machine, and what the packet asked or told.

6. **Look at what west-2 learned.** Stop the capture with Ctrl+C and read west-2's neighbour table:

```
# 108_L3_LAB_west-2
ip neigh
```

Question 3. west-2 now holds west-1's hardware address, and west-2 sent no request asking for it. Say where west-2 got it, naming the packet it came from and the fields of that packet it was read out of.

7. **Predict the effect of leaving the prefix length off.** Do not run this yet: write your answer first, then use `scripts/reset.sh` and try it if you want to check.

Question 4. Suppose you had typed `ip addr add 192.168.1.11 dev 108-WEST`, with no `/24`. `ip` supplies a prefix length when you omit one. State which length it supplies, how many addresses the resulting connected route covers, and whether the ping to 192.168.1.12 would still get a reply.

8. **Try to leave the subnet.** mid-1 is on Middle Net, which west-1 has no address on and no route to:

```
# 108_L3_LAB_west-1
ping -n -c 1 192.168.2.11
```

Question 5. Quote the message `ping` printed, exactly. Then say which machine produced that message and at what point: was a packet built and sent onto the wire and then rejected somewhere, or did the failure happen before anything was transmitted? Give the reason for your answer.

Part 2: address the whole network

Part 1 gave two machines an address each. Part 2 does the same for the remaining eight interfaces, in exactly the same way, and then looks at what that alone does and does not buy. Nothing in this part adds a route by hand; every route that appears is a connected route the kernel adds for you.

9. **Address the remaining eight interfaces** from the table in “The setup”. Six of them are the one interface on a host; the other two pairs are the two interfaces on each router. On a router, both interfaces get their address the same way, and the interface name is what tells them apart:

```
# 108_L3_LAB_west-router: two subnets, two interfaces, two commands each
docker exec -it 108_L3_LAB_west-router bash
ip addr add 192.168.1.1/24 dev 108-WEST
ip link set dev 108-WEST up
ip addr add 192.168.2.2/24 dev 108-MID
ip link set dev 108-MID up
ip addr show                                # both interfaces, both addresses, both UP
```

Work through mid-1, mid-2, east-1, east-2 and east-router the same way. When you think you are done, run **Status** in the TUI: its first stage lists all ten interfaces and ticks the ones that carry the right address and are up.

10. **Read a router’s table.** west-router has two addresses, so it has two connected routes and nothing else:

```
# 108_L3_LAB_west-router
ip route show
```

Question 6. Count the route entries the whole network holds at this point, across all six hosts and both routers. Show the arithmetic rather than only the total, and say what makes the two routers contribute a different number each from the hosts.

11. **Ping two addresses on the same machine.** 192.168.1.1 and 192.168.2.2 are both west-router. From west-1, one of them replies and one of them does not:

```
# 108_L3_LAB_west-1
ping -n -c 1 192.168.1.1      # west-router, West Net side
ping -n -c 1 192.168.2.2      # west-router, Middle Net side
```

Question 7. Both of those addresses belong to the same running machine, and the machine is up and forwarding. Explain why the second one fails anyway. Name the device that made the decision and the table it consulted to make it.

Part 3: a default route on every host

Every host can now reach its own subnet and nothing else, because a connected route is the only kind of route it holds. What a host needs next is not a description of the network. It needs

the address of one machine on its own subnet that will take a packet off its hands and deal with it, and a rule that says to hand over anything the host cannot deliver itself.

Definition: Next hop

The address a route sends a packet to, rather than the address the packet is ultimately for. It is written after **via** in an **ip route** command and printed in the **Gateway** column by **route -n**. A next hop must be an address the device can already reach without consulting another route, which in practice means an address on one of its own subnets: a route whose next hop is unreachable cannot be used to reach anything.

A default route is a route whose destination is 0.0.0.0/0, which matches every address there is, so it is the entry that gets used when no other entry matches. Its **next hop** is the router you are handing the packet to. For west-1 and west-2, only one machine on West Net is a router, so there is only one possible answer.

12. **Give each of the six hosts a default route.** Not the routers: a router in this lab gets specific routes in Part 4, and giving one a default as well would send anything it has no route for to the other router, which would send it straight back.

```
# 108_L3_LAB_west-1: and the same shape on the other five hosts
ip route add default via 192.168.1.1
ip route show                # two entries now: the connected route and the
                             default
```

West Net hosts point at 192.168.1.1; East Net hosts point at 192.168.3.1. Middle Net is the interesting one, and the next question is about it.

Question 8. Middle Net has two routers on it, so mid-1 has two addresses it could legally use as its default next hop. Give both. Say which one you configured, and state what makes the other one a legal choice too, given what each router is attached to.

13. **Find out what works now.** Ping between hosts and fill in this grid, one row per source:

From \ To	west-1	mid-1	east-1
west-1	—		
mid-1		—	
east-1			—

Question 9. Give your completed grid. Then take one pair from it that failed, quote the message the sending host printed, name the device that produced that message, and name the destination prefix that device had no entry for.

14. **Compare two failures that are not the same failure.** From west-1, ping mid-1 and then east-1:

```
# 108_L3_LAB_west-1
ping -n -c 1 192.168.2.11    # mid-1
ping -n -c 1 192.168.3.11    # east-1
```

Question 10. west-1 holds the same two routes for both of those destinations and used the same one for both, yet one gets a reply and the other does not. Explain the difference

in terms of what west-router holds, and say why the error for the second one arrives from 192.168.1.1 rather than being printed by west-1 the way the Part 1 failure was.

15. **Count the routers in a path without running traceroute.** Ping west-2 and then mid-1 from west-1, and read the `ttl=` field in each reply:

```
# 108_L3_LAB_west-1
ping -n -c 1 192.168.1.12      # west-2, no router in the path
ping -n -c 1 192.168.2.11      # mid-1, one router in the path
```

Question 11. Give the two TTL values. State which device changed the field and by how much, and say what TTL you would therefore expect in a reply from east-1 once the network is finished.

Part 4: static routes on the two routers

Every host now hands anything off its own subnet to a router, and West Net, Middle Net and East Net reach as far as the routers know how to take them. west-router is attached to West Net and Middle Net, so it has a connected route to each and no idea that East Net exists; east-router is the mirror image. Two routes close both gaps, one on each router.

Definition: Static route

A routing table entry an operator types, as opposed to a connected route the kernel derives from an interface address or a route a routing protocol learned from a neighbouring router. It names a destination prefix and a next hop, it stays in the table until somebody removes it, and it does not react to anything: if the path it names goes away, the entry stays and the packets keep going into the hole.

16. **Add the two static routes.** Each router needs one entry, for the subnet it is not attached to, pointing at the other router's Middle Net address:

```
# 108_L3_LAB_west-router: the one subnet this router cannot reach
ip route add 192.168.3.0/24 via 192.168.2.1
ip route show
```

east-router needs the matching entry for 192.168.1.0/24. Add nothing to the hosts: they already have a default route, and a host that also carries specific routes is a host you have to edit again every time the network changes.

Question 12. Give both commands you ran, with the router each was typed on. For each, explain why the next hop had to be an address on 192.168.2.0/24, and why naming an address on the destination subnet instead (192.168.3.1 on west-router, say) would not work.

Definition: Longest match

The rule that settles which route a device uses when more than one of them matches. The device first discards every route whose prefix does not match the destination, then, of the ones left, keeps only those with the longest prefix. A default route is not a fallback consulted after the others fail: it matches everything, so it survives the first step every time and is discarded in the second whenever any longer matching prefix is present. If the set ever empties, there is no route and the packet is discarded.

17. **Apply longest match to the smallest table in the lab.** mid-1 holds exactly two entries. Read them, then ask the kernel what it would do with two specific destinations:

```
# 108_L3_LAB_mid-1
ip route show
ip route get 192.168.2.12      # mid-2, on this host's own subnet
ip route get 192.168.3.11     # east-1, two subnets away
```

Question 13. For each of those two destinations, name the route mid-1 used. One of them matched both of mid-1's entries: say which one, and name the rule that discarded the other.

18. **Check the whole network.** From every host, every other host, and while you are there, all ten interfaces:

```
# 108_L3_LAB_west-1: and repeat from each of the other five hosts
ping -n -c 1 192.168.3.12      # east-2, the far corner
ping -n -c 1 192.168.2.1       # east-router, Middle Net side
```

Status does the full sweep for you: its last stage prints the thirty ordered pairs as a grid and reports how many of them reach each other. All thirty is the finish line.

Question 14. Count the route entries the finished network holds across all eight devices, and show the arithmetic. Then work out the count for a network built the other way, where every host carries a specific route to each of the two subnets it is not on instead of a default route, and the routers are unchanged. Give both totals, and say which one grows faster as more subnets are added.

Part 5: the router that names a better first hop

The network is finished, and one thing about it is still wrong. mid-1 sends everything off its own subnet to one router, but Middle Net has two, and for any destination in East Net the other one is a hop closer. Nothing in mid-1's configuration knows that. The router it is sending to does, and it says so.

Definition: ICMP redirect

A message a router sends back to the host that handed it a packet, when the router's own next hop for that packet sits on the same subnet as the host. It carries one address: the router the host should have used. RFC 792 requires the router to forward the packet anyway, so a redirect never costs the packet that provoked it; it is advice about the next one. The condition that triggers it is the router forwarding a packet back out of the interface it arrived on.

19. **Clear whatever mid-1 has already learned**, so the next packet to East Net is the first one the kernel has had to think about. Deleting and re-adding the default route is what does this; the usual `ip route flush cache` writes to `/proc/sys`, which Docker mounts read-only inside a container, and is refused:

```
# 108_L3_LAB_mid-1
ip route del default
ip route add default via 192.168.2.2
ip route get 192.168.3.11      # note the next hop it prints; you will compare
                               it later
```

20. **Watch Middle Net while mid-1 crosses it.** Start a capture on west-router's Middle Net interface, then ping east-1 three times from mid-1:

```
# 108_L3_LAB_west-router: leave this running
tcpdump -nevi 108-MID icmp
```

```
# 108_L3_LAB_mid-1: in your other terminal
ping -n -c 3 192.168.3.11
```

ping prints any **ICMP redirect** it receives on a line of its own, before the reply to the packet that provoked it. The capture shows the same messages on the wire, alongside the echo requests they were sent about.

Question 15. Give the number of redirects in the capture for that three-packet ping. Say which router sent it, which address it names, and why the second and third pings produced none.

21. **Ask mid-1 what it would do now.** Compare both of its tables with what you recorded before the ping:

```
# 108_L3_LAB_mid-1
ip route get 192.168.3.11      # the next hop for this one destination
ip route show                  # the routing table itself
```

Definition: Cached route exception

A per-destination override the kernel attaches beside a route, rather than an entry in the routing table. A redirect installs one, for the single destination the redirect named, with a lifetime of about five minutes. `ip route show` does not print it, because the table has not changed; `ip route get` does, marked `<redirected>` and with the time left before it expires.

What the redirect left behind is a **cached route exception**, and the two commands above disagree about mid-1 because only one of them reports one.

Question 16. Give the next hop `ip route get` printed before the ping and the next hop it prints now, together with the time the entry says it has left. Then predict what `ip route get` will report for 192.168.3.11 once you delete and re-add mid-1's default route, run those two commands, and say whether your prediction held.

22. **Draw the first ping.** The capture shows the packets that crossed west-router's own Middle Net interface, which is not everything Middle Net carried: the switch delivers a unicast frame only to the port that owns the destination MAC address. The `ttl` field on each captured packet says how far it has travelled, and for the echo reply the `ttl=` value `ping` printed on mid-1 says the same.

Question 17. Draw the first of the three pings as a time sequence diagram, with a vertical line for each of mid-1, west-router, east-router and east-1, and one labelled arrow for every packet: the echo request on each hop it takes, the redirect, and the echo reply. Mark which arrows crossed Middle Net.

23. **Weigh the two fixes.** The redirect corrected mid-1's path to east-1 without anyone touching mid-1's configuration. Changing mid-1's default gateway to 192.168.2.1 would fix it a different way.

Question 18. Give one cost of leaving mid-1's default gateway as it is and letting the redirect do the work. Base your answer on two things you have measured: which packets of the ping took the longer path, and how long the cached route exception said it had left.

Troubleshooting

Work down this list before asking for help, in this order, because each step assumes the ones above it are right.

Check the interface. `ip link show 108-WEST` has to say UP, and `ip addr show 108-WEST` has to show the address *and* the prefix length you meant. An interface with an address and no `ip link set ... up` is the most common single fault in Part 1, and an interface with the right address and /32 instead of /24 is the most common second one.

Read the error, because the two errors mean different things. `ping: connect: Network unreachable` is your own machine refusing to build the packet, because nothing in its routing table matched; fix the sender. `From 192.168.1.1 icmp_seq=1 Destination Net Unreachable` means the packet was built, sent, and refused by the machine named at the start of the line; go and fix the routing table on that machine, not on the sender.

Remember the reply has to get home too. A ping needs a route in both directions. If A can reach B and B cannot reach A, the fault is on the return path, and the return path may not be the same devices as the forward path.

Check the routing table, then the specific destination. `ip route show` lists what the device holds; `ip route get <address>` says which of those entries it would actually use for one address, which is faster than applying longest match in your head.

Watch the wire when the tables look right. `tcpdump -nevi <interface>` on the device nearest the sender says whether the packet is arriving at all. If you see ARP requests and nothing else, the sender believes the destination is on its own subnet and no machine there is replying, which usually means a prefix length is wrong. If you see the echo request arrive at a router and nothing leave it, that router has no route for the destination.

Undoing things. `ip addr del 192.168.1.11/24 dev 108-WEST` removes an address and `ip route del 192.168.3.0/24` removes a route. `RTNETLINK answers: File exists` means the address or route you are adding is already there. If a device is beyond repair, **Reset to baseline** returns the whole lab to its unconfigured state and you can build it again.

Checks: what “done” means

- **Every interface:** all ten carry the address in the setup table and report UP.
- **Every subnet:** each device reaches every other device on its own subnet, including the router interfaces there.
- **Every host:** exactly two route entries, the connected one and a default whose next hop is a router on its own subnet. Neither router has a default route.
- **Every pair:** all thirty ordered pairs of the six hosts reach each other.

`scripts/status.sh` checks all four and prints the last one as a grid, so you can read how far you have got without shelling in. To start again, `scripts/reset.sh` deletes every address and route you configured and leaves the network wired and blank. `scripts/teardown.sh` removes the lab.

Safety. Everything here lives on three isolated switches with no route off the host. The addresses are private ones from RFC 1918, nothing in the lab is reachable from outside it, and no packet you send can leave it.

When you are done, tear the lab down. Choose **Teardown** in the TUI, or run `scripts/teardown.sh` directly, to remove the containers and their wiring so nothing is left running on your host. **Reset to baseline** only wipes your configuration; only **Teardown** deletes the lab.